SPRINGBOOT DEVTOOLS

eazy bytes

- *The Spring Boot DevTools provides features like Automatic restart & LiveReload that make the application development experience a little more pleasant for developers.*
- *It can be added into any of the SpringBoot project by adding the below maven dependency,*

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

DevTools maintains 2 class loaders. one with classes that doesn't change and other one with classes that change. When restart needed it only reload the second class loader which makes restarts faster as well.

DevTools includes an embedded LiveReload server that can be used to trigger a browser refresh when a resource is changed. LiveReload related browser extensions are freely available for Chrome, Firefox

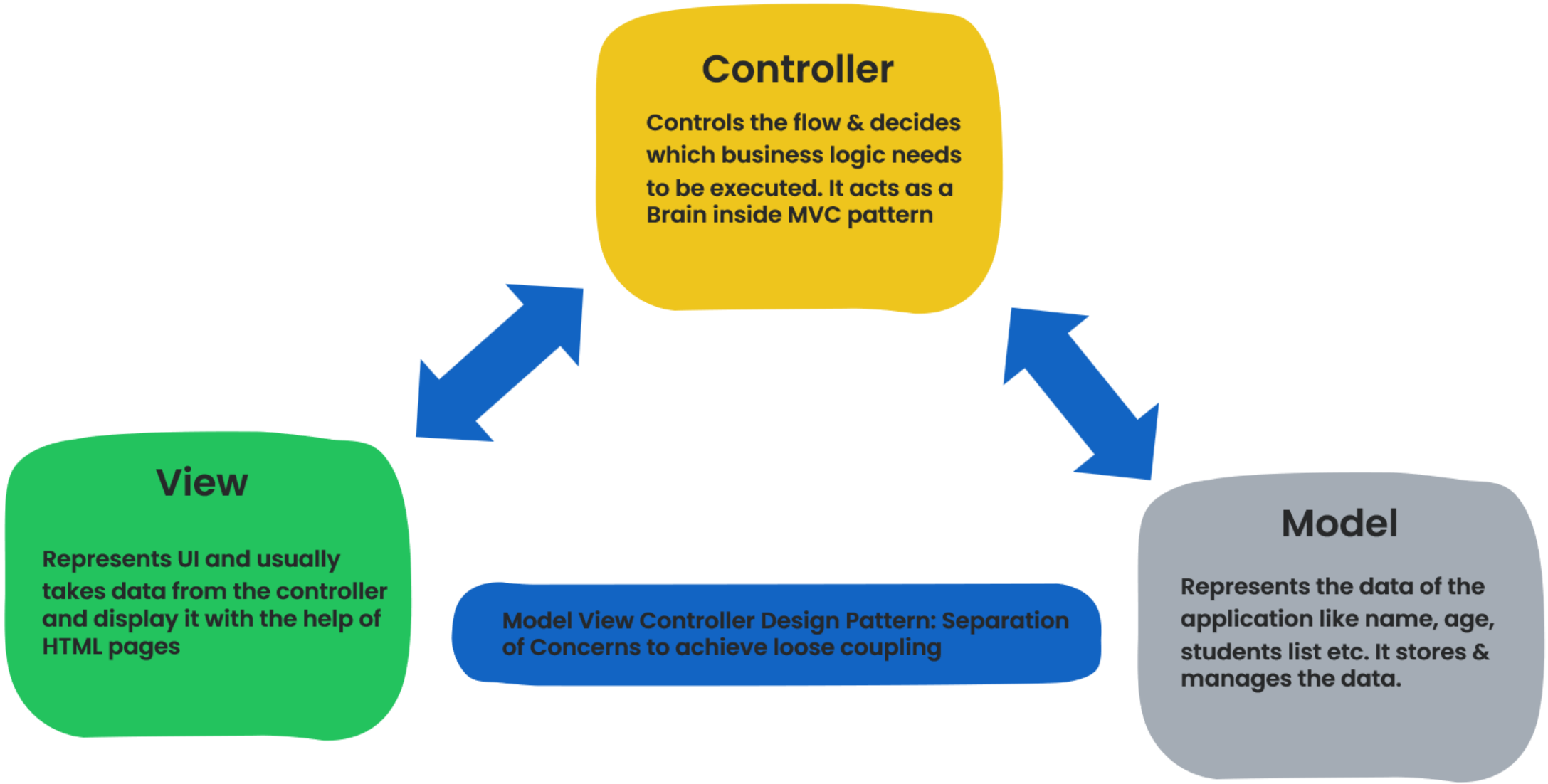
DevTools triggers a restart when ever a build is triggered through IDE or by maven commands etc.

DevTools disables the caching options by default during development.

Repackaged archives do not contain DevTools by default.

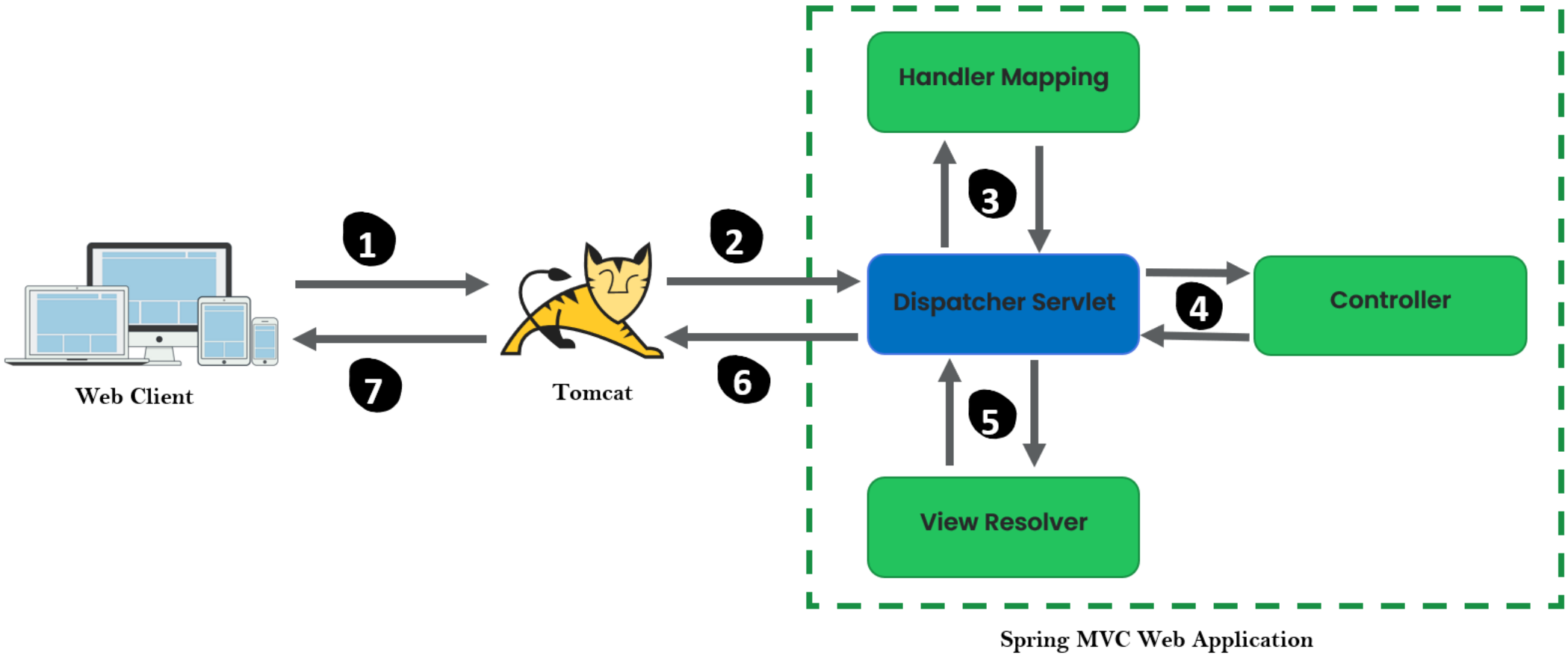
INTRODUCTION TO MVC PATTERN

eazy bytes



SPRING MVC ARCHITECTURE & INTERNAL FLOW

eazy bytes



SPRING MVC ARCHITECTURE & INTERNAL FLOW

eazy bytes

1. Web Client makes HTTP request

2. Servlet Containers like Tomcat accepts the HTTP requests and handovers the Servlet Request to Dispatcher Servlet inside Spring Web App.

3. The Dispatcher Servlet will check with the Handler Mapping to identify the controller and method names to invoke based on the HTTP method, path etc.

4. The Dispatcher Servlet will invoke the corresponding controller & method. After execution, the controller will provide a view name and data that needs to be rendered in the view.

5. The Dispatcher Servlet with the help of a component called View Resolver finds the view and render it with the data provided by the controller.

6. The Servlet Container or Tomcat accepts the Servlet Response from the Dispatcher servlet and convert the same to HTTP response before returning to the client.

7. The browser or client intercepts the HTTP response and display the view, data etc.

QUICK TIP

eazy bytes

Do you know, we can register view controllers that create a direct mapping between the URL and the view name using the ViewControllerRegistry. This way, there's no need for any Controller between the two.

```java
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController( urlPathOrPattern: "/courses").setViewName("courses");
        registry.addViewController( urlPathOrPattern: "/about").setViewName("about");
    }

}
```

REDUCING BOILERPLATE CODE WITH LOMBOK

eazy bytes

- *Java expects lot of boilerplate code inside POJO classes like getters and setters.*
- *Lombok, which is a Java library provides you with several annotations aimed at avoiding writing Java code known to be repetitive and/or boilerplate.*
- *It can be added into any of the Java project by adding the below maven dependency,*

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
</dependency>
```

Project Lombok works by plugging into your build process. Then, it will auto-generate the Java bytecode into your .class files required to implement the desired behavior, based on the annotations you used.

Most commonly used Lombok annotations,

@Getter, @Setter
@NoArgsConstructor
@RequiredArgsConstructor
@AllArgsConstructor
@ToString, @EqualsAndHashCode
@Data

@Data is a shortcut annotation that combines the features of below annotations together,

@ToString
@EqualsAndHashCode
@Getter, @Setter
@RequiredArgsConstructor

REDUCING BOILERPLATE CODE WITH LOMBOK

eazy bytes

```java
/*
@Data annotation is provided by Lombok library which generates getter, setter,
equals(), hashCode(), toString() methods & Constructor at compile time.
This makes our code short and clean.
* */
@Data
public class Contact {

    private String name;
    private String mobileNum;
    private String email;
    private String subject;
    private String message;
}
```

Contact.java

Inherited members (Ctrl+F12) Anonymous Classes (Ctrl+I) Lambdas (Ctrl+L)

- Contact
  - Contact()
  - canEqual(Object): boolean
  - equals(Object): boolean ↑Object
  - getEmail(): String
  - getMessage(): String
  - getMobileNum(): String
  - getName(): String
  - getSubject(): String
  - hashCode(): int ↑Object
  - setEmail(String): void
  - setMessage(String): void
  - setMobileNum(String): void
  - setName(String): void
  - setSubject(String): void
  - toString(): String ↑Object
  - email: String
  - message: String
  - mobileNum: String
  - name: String
  - subject: String

A sample outline of a Java POJO class with @Data annotation used. The source code will not have boilerplate code but the compiled byte code will have.

@RequestParam Annotation

eazy bytes

- *In Spring @RequestParam annotation is used to map either query parameters or form data.*
- *For example, if we want to get parameters value from a HTTP GET requested URL then we can use @RequestParam annotation like in below example.*

http://localhost:8080/holidays?festival=true&federal=true

```
@GetMapping("/holidays")
public String displayHolidays(@RequestParam(required = false) boolean festival,
                              @RequestParam(required = false) boolean federal) {
    // Business Logic
    return "holidays.html";
}
```

✓ The @RequestParam annotation supports attributes like name, required, value, defaultvalue. We can use them in our application based on the requirements.

✓ The name attribute indicates the name of the request parameter to bind to.

✓ The required attribute is used to make a field either optional or mandatory. If it is mandatory, an exception will throw in case of missing fields.

✓ The value attribute is similar to name elements and can be used as an alias.

✓ defaultValue for the parameter is to handle missing values or null values. If the parameter does not contain any value then this default value will be considered.

@PathVariable Annotation

eazy bytes

- *The @PathVariable annotation is used to extract the value from the URI. It is most suitable for the RESTful web service where the URL contains some value. Spring MVC allows us to use multiple @PathVariable annotations in the same method.*
- *For example, if we want to get the value from a requested URI path, then we can use @PathVariable annotation like in below example.*

http://localhost:8080/holidays/all
http://localhost:8080/holidays/federal
http://localhost:8080/holidays/festival

```
@GetMapping("/holidays/{display}")
public String displayHolidays(@PathVariable String display) {
    //Business Logic
    return "holidays.html";
}
```

✓ **The @PathVariable annotation supports attributes like name, required, value similar to @RequestParam. We can use them in our application based on the requirements.**

VALIDATION WITH SPRING BOOT

eazy bytes

- *Bean Validation (https://beanvalidation.org/) is the standard for implementing validations in the Java ecosystem. It's well integrated with Spring and Spring Boot.*
- *Below is the maven dependency that we can add to implement Bean validations in any Spring/SpringBoot project,*

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Bean Validation works by defining constraints to the fields of a class by annotating them with certain annotations.

We can put the @Valid annotation on method parameters and fields to tell Spring that we want a method parameter or field to be validated.

Below are the important packages where validations related annotations can be identified,

- ✓ jakarta.validation.constraints.*
- ✓ org.hibernate.validator.constraints.*

eazy bytes

Important Validation Annotations

jakarta.validation.constraints.*

- ✓ @Digits
- ✓ @Email
- ✓ @Max
- ✓ @Min
- ✓ @NotBlank
- ✓ @NotEmpty
- ✓ @NotNull
- ✓ @Pattern
- ✓ @Size

org.hibernate.validator.constraints.*

- ✓ @CreditCardNumber
- ✓ @Length
- ✓ @Currency
- ✓ @Range
- ✓ @URL
- ✓ @UniqueElements
- ✓ @EAN
- ✓ @ISBN

VALIDATION WITH SPRING BOOT

eazy bytes

- *Sample validations declaration inside a java POJO class,*

```java
@Data
public class Contact {

    @NotBlank(message="Email must not be blank")
    @Email(message = "Please provide a valid email address" )
    private String email;

    @NotBlank(message="Subject must not be blank")
    @Size(min=5, message="Subject must be at least 5 characters long")
    private String subject;

    @NotBlank(message="Message must not be blank")
    @Size(min=10, message="Message must be at least 10 characters long")
    private String message;
}
```

VALIDATION WITH SPRING BOOT

eazy bytes

- *We can put the @Valid annotation on method parameters to tell Spring framework that we want a particular POJO object needs to be validated based on the validation annotation configurations. For any issues, framework populates the error details inside the Errors object. The errors can be used to display on the UI to the user. Sample example code is below,*

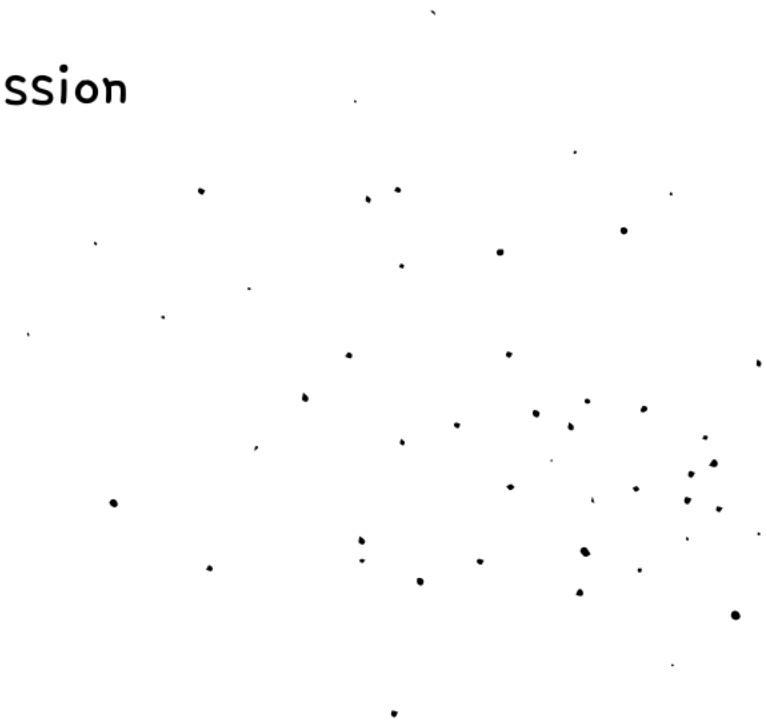
```
@RequestMapping(value = "/saveMsg",method = POST)
public String saveMessage(@Valid @ModelAttribute("contact") Contact contact, Errors errors) {

    if(errors.hasErrors()){
        log.error("Contact form validation failed due to : " + errors.toString());
        return "contact.html";
    }
    contactService.saveMessageDetails(contact);
    return "redirect:/contact";
}
```

```
<ul>
    <li class="alert alert-danger" role="alert" th:each="error : ${#fields.errors('contact.*')}"
        th:text="${error}" />
</ul>
```

eazy bytes

Bean Scopes inside Spring

1. Singleton
2. Prototype
3. Request
4. Session
5. Application



SPRING WEB SCOPES

eazy bytes

Web Scopes inside Spring

- ✓ **Request (@RequestScope)**
- ✓ **Session (@SessionScope)**
- ✓ **Application (@ApplicationScope)**

1. **Request Scope** - Spring creates an instance of the bean class for every HTTP request. The instance exists only for that specific HTTP request.
2. **Session Scope** - Spring creates an instance and keeps the instance in the server's memory for the full HTTP session. Spring links the instance in the context with the client's session.
3. **Application Scope** - The instance is unique in the app's context, and it's available while the app is running.

eazy bytes

Key points of Spring Web scopes

Request Scope

- ✓ Spring creates a lot of instances of this bean in the app's memory for each HTTP request. So these type of beans are short lived.
- ✓ Since Spring creates a lot of instances, please make sure to avoid time consuming logic while creating the instance.
- ✓ Can be considered for the scenarios where the data needs to be reset after new request or page refresh etc..

Session Scope

- ✓ Session scoped beans have longer life & they are less frequently garbage collected.
- ✓ Avoid keeping too much information inside session data as it impacts performance. Never store sensitive information as well.
- ✓ Can be considered for the scenarios where the same data needs to be accessed across multiple pages like user information.

Application Scope

- ✓ In the application scope, Spring creates a bean instance per web application runtime.
- ✓ It is similar to singleton scope, with one major difference. Singleton scoped bean is singleton per ApplicationContext where application scoped bean is singleton per ServletContext.
- ✓ Can be considered for the scenarios where we want to store Drop Down values, Reference table values which won't change for all the users.

SPRING SECURITY

eazy bytes

- *Spring Security is a powerful and highly customizable authentication and access-control framework. It is the de-facto standard for securing Spring-based applications.*
- *Below is the maven dependency that we can add to implement security using Spring Security project in any of the SpringBoot projects,*

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Spring Security is a framework that provides authentication, authorization, and protection against common attacks.

Spring Security helps developers with easier configurations to secure a web application by using standard username/password authentication mechanism.

Spring Security provides out of the box features to handle common security attacks like CSRF, CORs. It also has good integration with security standards like JWT, OAUTH2 etc.

AUTHENTICATION & AUTHORIZATION

eazy bytes

AUTHENTICATION

- In authentication, the identity of users are checked for providing the access to the system.
- Authentication done before authorization
- It needs usually user's login details
- If authentication fails usually we will get 401 error response
- For example as a Bank customer/employee in order to perform actions in the app, we need to prove our identity

AUTHORIZATION

- In authorization, person's or user's authorities are checked for accessing the resources.
- Authorization always happens after authentication.
- it needs user's privilege or roles
- If authorization fails usually we will get 403 error response
- Once logged into the application, my roles, authorities will decide what kind of actions I can do

QUICK TIP

eazy bytes

Do you know,

- ✓ **As soon as we add spring security dependency to a web application, be default it protects all the pages/APIs inside it. It will redirect to the inbuilt login page to enter credentials.**
- ✓ **The default credentials are user and password is randomly generated & printed on the console.**
- ✓ **We can configure custom credentials using the below properties to get started for POCs etc. But for PROD applications, Spring Security supports user credentials configuration inside DB, LDAP, OAUTH2 Server etc.**

```
spring.security.user.name = eazybytes
spring.security.user.password = 12345
```

DEFAULT SECURITY CONFIGURATIONS IN SPRING SECURITY

eazy bytes

By default, Spring Security framework protects all the paths present inside the web application. This behaviour is due to the code present inside the method defaultSecurityFilterChain(HttpSecurity http) of class SpringBootWebSecurityConfiguration

```
@Bean
@Order(SecurityProperties.BASIC_AUTH_ORDER)
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests((requests) -> requests.anyRequest().authenticated());
    http.formLogin(withDefaults());
    http.httpBasic(withDefaults());
    return http.build();
}
```

CONFIGURE permitAll() WITH SPRING SECURITY

eazy bytes

- *Using permitAll() configurations we can allow full/public access to a specific resource/path or all the resources/paths inside a web application.*
- *Below is the sample configuration that we can do in order to allow any requests in a Web application with out security,*

```
@Configuration
public class ProjectSecurityConfig {

  @Bean
  SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {

    http.authorizeHttpRequests((requests) -> requests.anyRequest().permitAll())
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults());
    return http.build();

  }
```

- Form Login provides support for username and password being provided through an html form
- HTTP Basic Auth uses an HTTP header in order to provide the username and password when making a request to a server.

CONFIGURE denyAll() WITH SPRING SECURITY

eazy bytes

- *Using denyAll() configurations we can deny access to a specific resource/path or all the resources/paths inside a web application regardless of user authentication.*
- *Below is the sample configuration that we can do in order to deny any requests that is coming into a web application,*

```
@Configuration
public class ProjectSecurityConfig {

  @Bean
  SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {

    http.authorizeHttpRequests((requests) -> requests.anyRequest().denyAll())
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults());
    return http.build();

  }
```

- Usually denyAll() is used to retire a specific API temporarily with out removing the code
- permitAll() is used to allow public access to public APIs, paths, CSS, images, JS files etc.

CONFIGURE CUSTOM SECURITY CONFIGs & CSRF DISABLE

eazy bytes

- *We can apply custom security configurations based on our requirements for each API/URL like below.*
- *permitAll() can be used to allow access w/o security and authenticated() can be used to protect a web page/API.*
- *By default any requests with HTTP methods that can update data like POST, PUT will be stopped with 403 error due to CSRF protection. We can disable the same for now and enable it in the coming sections when we started generating CSRF tokens.*
- *Below is the sample configuration that we can do to implement custom security configs and disable CSRF,*

```
@Configuration
public class ProjectSecurityConfig {

   @Bean
   SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
      http.csrf((csrf) -> csrf.disable())
            .authorizeHttpRequests((requests) -> requests
               .requestMatchers("", "/", "/home").permitAll()
               .requestMatchers("/holidays/**").permitAll()
               .requestMatchers("/contact").permitAll()
               .requestMatchers("/saveMsg").permitAll()
               .requestMatchers("/courses").permitAll()
               .requestMatchers("/about").permitAll()
               .requestMatchers("/assets/**").permitAll())
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());
      return http.build();
   }
```

IN-MEMORY AUTHENTICATION IN SPRING SECURITY

eazy bytes

- *Spring Security provide support for username/password based authentication based on the users stored in application memory.*
- *Like mentioned below, we can configure any number of users & their roles, passwords using in-memory authentication,*

```java
@Configuration
public class ProjectSecurityConfig {

  @Bean
  SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {...}

  @Bean
  public InMemoryUserDetailsManager userDetailsService() {

    UserDetails user = User.withDefaultPasswordEncoder()
        .username("user").password("12345").roles("USER").build();

    UserDetails admin = User.withDefaultPasswordEncoder()
        .username("admin").password("54321").roles("USER", "ADMIN").build();
    return new InMemoryUserDetailsManager(user, admin);
  }

}
```

in-memory authentication is idle for POC Web Apps or any internal Web App that get used only in non-prod environments. NEVER EVER use in-memory authentication for PROD web applications.

CONFIGURING LOGIN & LOGOUT PAGE

eazy bytes

- *Spring Security allows us to configure a custom login page to our web application instead of using the Spring Security default provided login page.*
- *Similarly we can configure logout page as well.*
- *Below is the sample configuration that we can follow,*

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http.csrf((csrf) -> csrf.disable())
            .authorizeHttpRequests((requests) -> requests.requestMatchers("/dashboard").authenticated()
            .requestMatchers("", "/", "/home").permitAll()
            .requestMatchers("/holidays/**").permitAll()
            .requestMatchers("/contact").permitAll()
            .requestMatchers("/saveMsg").permitAll()
            .requestMatchers("/courses").permitAll()
            .requestMatchers("/about").permitAll()
            .requestMatchers("/login").permitAll()
            .requestMatchers("/assets/**").permitAll())
            .formLogin(loginConfigurer -> loginConfigurer.loginPage("/login")
                    .defaultSuccessUrl("/dashboard").failureUrl("/login?error=true").permitAll())
            .logout(logoutConfigurer -> logoutConfigurer.logoutSuccessUrl("/login?logout=true")
                    .invalidateHttpSession(true).permitAll())
            .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Configured login page will be shown, if the user tries to access the secured page/resource with out a valid authenticated session. The same behavior applies for default login page provided by Spring Security

QUICK TIP

eazy bytes

Do you know, Thymeleaf has a great integration with Spring Security. More details can be found at https://www.thymeleaf.org/doc/articles/springsecurity.html

Step 1 : Add the below dependency in the pom.xml

```
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

Step 2 : Add the below XML name space which enable us to use Thymeleaf Security related tags

```
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
```

Step 3 : Use any of the Thymeleaf Security tags inside HTML code based on Authentication details,

sec:authorize="isAnonymous()" , sec:authorize="isAuthenticated()" , sec:authorize="hasRole('ROLE_ADMIN')"
sec:authentication="name" , sec:authentication="principal.authorities"

@ControllerAdvice & @ExceptionHandler

eazy bytes

- *@ControllerAdvice is a specialization of the @Component annotation which allows to handle exceptions across the whole application in one global handling component. You can think of it as an interceptor of exceptions thrown by methods annotated with @RequestMapping or one of the shortcuts like @GetMapping etc.*
- *We can define the exception handle logic inside a method and annotate it with @ExceptionHandler*
- *Below is the sample configuration that we can follow,*

```java
@ControllerAdvice
public class GlobalExceptionController {

    @ExceptionHandler(Exception.class)
    public ModelAndView exceptionHandler(Exception exception){
        ModelAndView errorPage = new ModelAndView();
        errorPage.setViewName("error");
        errorPage.addObject("errormsg", exception.getMessage());
        return errorPage;
    }

}
```

Combination of @ControllerAdvice & @ExceptionHandler can handle the exceptions across all the controllers inside a web application globally.

QUICK TIP

eazy bytes

Do you know,

- ✓ If a method annotated with `@ExceptionHandler` present inside a `@Controller` class, then the exception handling logic will be applicable for any exceptions occurred in that specific controller class.
- ✓ If the same `@ExceptionHandler` annotated method present inside a `@ControllerAdvice` class, then the exception handling logic will be applicable for any exceptions occurred across all the controller classes.
- ✓ Using `@ExceptionHandler` annotation, we can handle any number of exceptions like below sample code,

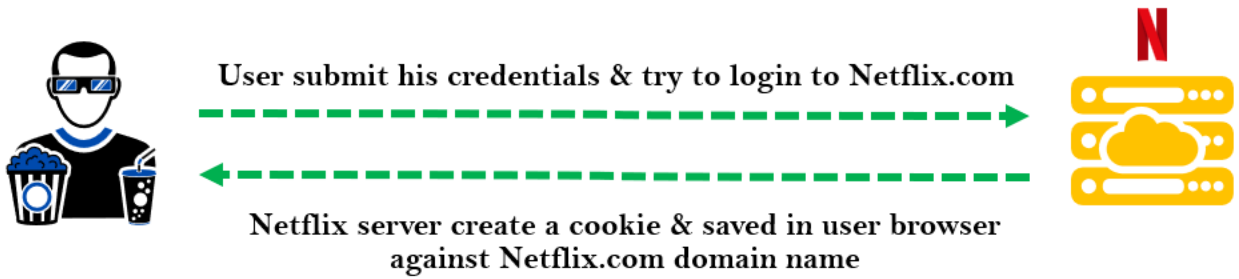
```
@ExceptionHandler({NullPointerException.class,
                   ArrayIndexOutOfBoundsException.class,
                   IOException.class})
public ModelAndView handleException(RuntimeException ex)
{
    //Exception handling logic
}
```

CROSS-SITE REQUEST FORGERY (CSRF)

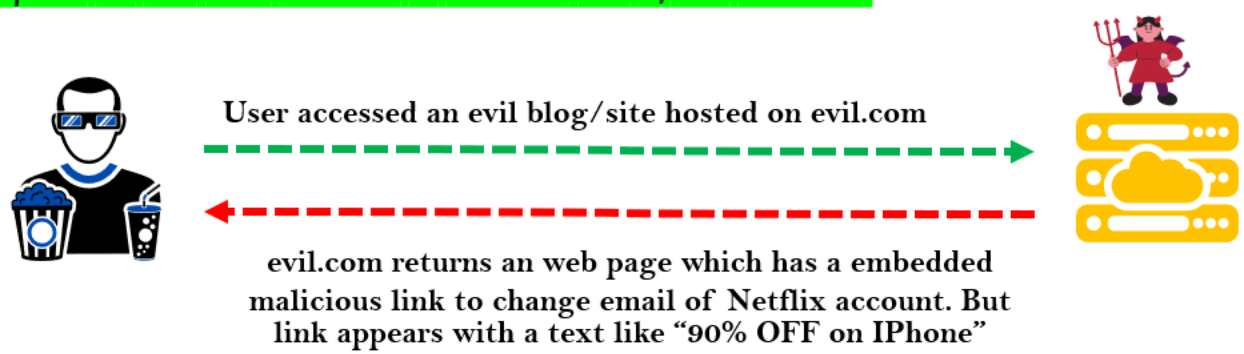
eazy bytes

- *A typical Cross-Site Request Forgery (CSRF or XSRF) attack aims to perform an operation in a web application on behalf of a user without their explicit consent. In general, it doesn't directly steal the user's identity, but it exploits the user to carry out an action without their will.*
- *Consider you are using a website netflix.com and the attacker's website evil.com.*

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com



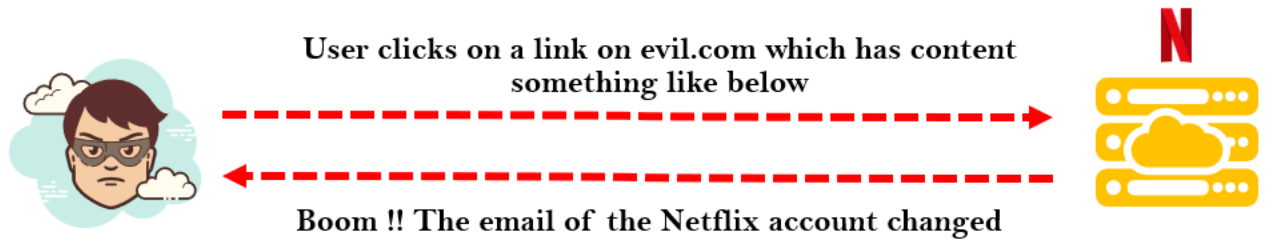
Step 2 : The same Netflix user opens an evil.com website in another tab of the browser.



CROSS-SITE REQUEST FORGERY (CSRF)

eazy bytes

Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com, the backend server of Netflix.com can't differentiate from where the request came. So here the evil.com forged the request as if it is coming from a Netflix.com UI page.




```
<form action="https://netflix.com/changeEmail"
  method="POST" id="form">
    <input type="hidden" name="email" value="user@evil.com">
<form>

<script>
    document.getElementById('form').submit()
</script>
```

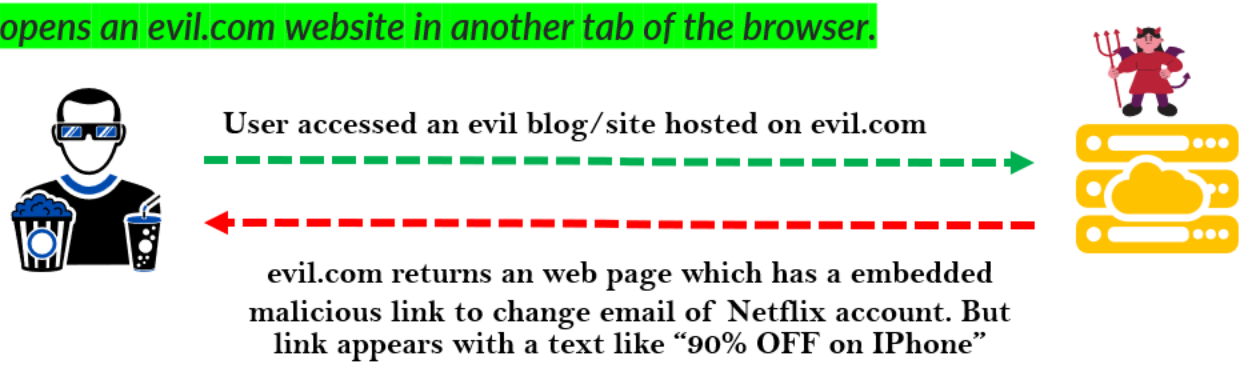
SOLUTION TO CSRF

eazy bytes

- *To defeat a CSRF attack, applications need a way to determine if the HTTP request is legitimately generated via the application's user interface. The best way to achieve this is through a CSRF token. A CSRF token is a secure random token that is used to prevent CSRF attacks. The token needs to be unique per user session and should be of large random value to make it difficult to guess.*
- *Let's see how this solve CSRF attack by taking the previous Netflix example again,*

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com along with a randomly generated unique CSRF token for this particular user session. CSRF token is inserted within hidden parameters of HTML forms to avoid exposure to session cookies.

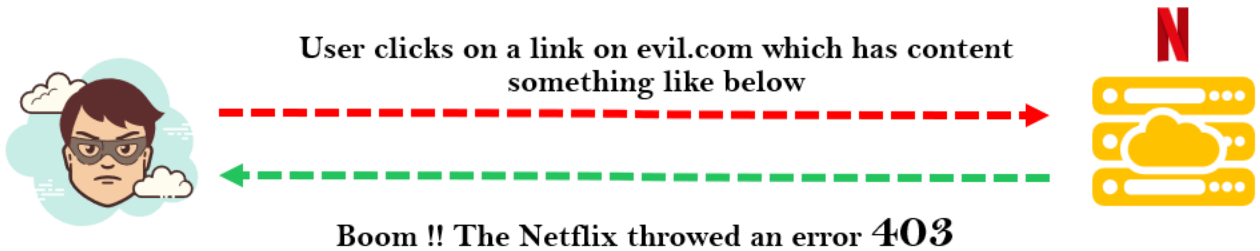
Step 2 : The same Netflix user opens an evil.com website in another tab of the browser.



SOLUTION TO CSRF

eazy bytes

Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com. This time the Netflix.com backend server expects CSRF token along with the cookie. The CSRF token must be same as initial value generated during login operation



The CSRF token will be used by the application server to verify the legitimacy of the end-user request if it is coming from the same App UI or not. The application server rejects the request if the CSRF token fails to match the test.

QUICK TIP

eazy bytes

Do you know,

- ✓ **By default, Spring Security enables CSRF fix for all the HTTP methods which results in data change like POST, DELETE etc. But not for GET.**

- ✓ **Using Spring Security configurations we can disable the CSRF protection for complete application or for only few paths based on our requirements like below.**

  - ***http.csrf((csrf) -> csrf.disable())***
  - ***http.csrf((csrf) -> csrf.ignoringRequestMatchers("/saveMsg"))***

- ✓ **Thymeleaf has great integration & support with Spring Security to generate a CSRF token. We just need to add the below code in login html form code and Thymeleaf will automatically appends the CSRF token for the remaining pages/forms inside the web application,**

```
<input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}" />
```

SPRING BOOT & H2 DATABASE

eazy bytes

- *H2 is an embedded, open-source, and in-memory database. SpringBoot supports integration with H2 DB which can be used for POC applications and excellent for examples/testing.*
- *Below is the maven dependency that we can add to any SpringBoot projects in order to use internal memory H2 Database,*

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Since it is a internal memory DB, we need to create the schema and data that is needed during startup of the App. Any updates to the data will be lost after restarting the server.

To create schema & data for the H2 DB, we can add schema.sql & data.sql inside the maven project's resources folder. Any table creation scripts and DB records scripts can be present inside schema.sql and data.sql respectively.

By default, the H2 web console is available at /h2-console. You can customize the console's path by using the spring.h2.console.path property.

The default credentials of H2 DB are username is sa and password is "" (Empty)

eazy bytes

Key points of JDBC

Intro to JDBC

- ✓JDBC or Java Database Connectivity is a specification from Core Java that provides a standard abstraction for java apps to communicate with various databases.
- ✓JDBC API along with the database driver is capable of accessing databases
- ✓JDBC is a base framework or standard for frameworks like Hibernate, Spring Data JPA, MyBatis etc.

Steps in JDBC to access DB

We need to follow the below steps to access DB using JDBC,

1) Load Driver Class
2) Obtain a DB connection
3) Obtain a statement using connection object
4) Execute the query
5) Process the result set
6) Close the connection

Problem with JDBC

- ✓Developers are forced to follow all the steps mentioned to perform any kind of operation with DB which results in lot of duplicate code at many places.
- ✓Developers needs to handle the checked exceptions that will throw from the API.
- ✓JDBC is database dependent

INTRO TO JDBC & PROBLEMS WITH IT

eazy bytes

```
public Optional<Contact> findById(int id) {
    Connection connection = null;
    PreparedStatement statement = null;
    ResultSet resultSet = null;
    try {
        Class.forName("com.mysql.jdbc.Driver");
        connection = DriverManager.getConnection("url","username","pwd");
        statement = connection.prepareStatement(
                "select id, name, message from contact");
        statement.setInt(1, id);
        resultSet = statement.executeQuery();
        Contact contact = null;
        if(resultSet.next()) {
            contact = new Contact(
                    resultSet.getInt("id"),
                    resultSet.getString("name"),
                    resultSet.getString("message"));
        }
        return Optional.of(contact);
    } catch (SQLException e) {
        // ??? What should be done here ???
    } finally {
        if (resultSet != null) {
            try {
                resultSet.close();
            } catch (SQLException e) {}
        }
        if (statement != null) {
            try {
                statement.close();
            } catch (SQLException e) {}
        }
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {}
        }
    }
    return null;
}
```

Sample program using JDBC to fetch a record from the database table. You can see there is lot of boilerplate code present

JDBC Program.txt

INTRO TO SPRING JDBC

eazy bytes

- *Spring JDBC simplifies the use of JDBC and helps to avoid common errors. It executes core JDBC workflow, leaving application code to provide SQL and extract results. It does this magic by providing JDBC templates which developers can use inside their applications.*
- *Below is the maven dependency that we need to add to any Spring/SpringBoot projects in order to use Spring JDBC provided templates,*

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
```

Spring provides many templates for JDBC related activities. Among them, the famous ones are JdbcTemplate , NamedParameterJdbcTemplate.

JdbcTemplate is the classic and most popular Spring JDBC approach. This provides "lowest-level" approach and all others templates uses JdbcTemplate under the covers.

NamedParameterJdbcTemplate wraps a JdbcTemplate to provide named parameters instead of the traditional JDBC ? placeholders. This approach provides better documentation and ease of use when you have multiple parameters for an SQL statement.

Spring JDBC - who does what?

Action	Spring JDBC	Developer
Define connection parameters		✕
Open the connection	✕	
Specify the SQL statement		✕
Declare parameters and provide parameter values		✕
Prepare and run the statement	✕	
Set up the loop to iterate through the results (if any).	✕	
Do the work for each iteration		✕
Process any exception	✕	
Handle transactions	✕	
Close the connection, the statement, and the resultset	✕	

USING JdbcTemplate

eazy bytes

- *JdbcTemplate is the central class in the JDBC core package. It handles the creation and release of resources, which helps you avoid common errors, such as forgetting to close the connection. It performs the basic tasks of the core JDBC workflow (such as statement creation and execution), leaving application code to provide SQL and extract results.*

- *You can use JdbcTemplate within a DAO implementation through direct instantiation with a DataSource reference, or you can configure it in a Spring IoC container and give it to DAOs as a bean reference.*

- *We need to follow the below steps in order to configure JdbcTemplate inside a Spring Web application (With out Spring Boot)*

Step 1 : First we need to create a Data Source Bean inside Web application with the DB credentials like mentioned below.

```
@Bean
public DataSource myDataSource() {
    DriverManagerDataSource dataSource = new DriverManagerDataSource();
    dataSource.setDriverClassName("com.mysql.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/eazyschool");
    dataSource.setUsername("user");
    dataSource.setPassword("password");
    return dataSource;
}
```

USING JdbcTemplate

eazy bytes

Step 2 : Inside any Repository/DAO classes where we want to execute queries, we need to create a bean/object of JdbcTemplate by injecting data source bean

```
@Repository
public class PersonDAOImpl implements PersonDAO {
    JdbcTemplate jdbcTemplate;

    @Autowired
    public PersonDAOImpl(DataSource dataSource) {
        jdbcTemplate = new JdbcTemplate(dataSource);
    }

}
```

- *Instances of the JdbcTemplate class are thread-safe, once configured. This is important because if needed we can configure a single instance of a JdbcTemplate and then safely inject this shared reference into multiple DAOs (or repositories).*

USING JdbcTemplate

eazy bytes

Sample usage of JdbcTemplate for SELECT queries,

```
// The following query gets the number of rows in a table
int rowCount = this.jdbcTemplate.queryForObject("select count(*) from person", Integer.class);
```

```
// The following query uses a bind variable
int countOfPersonsNamedJoe = this.jdbcTemplate.queryForObject(
        "select count(*) from person where first_name = ?", Integer.class, "Joe");
```

```
// The following query looks for a string column based on a condition:
String lastName = this.jdbcTemplate.queryForObject("select last_name from person where id = ?",
        String.class, 1212L);
```

USING JdbcTemplate

eazy bytes

Sample usage of JdbcTemplate for Updating (INSERT, UPDATE, and DELETE)

```
// The following example inserts a new entry
this.jdbcTemplate.update("insert into person (first_name, last_name) values (?, ?)",
        "Jon", "Doe");
```

```
// The following example updates an existing entry
this.jdbcTemplate.update("update person set last_name = ? where id = ?",
        "Maria", 5276L);
```

```
// The following example deletes an entry
this.jdbcTemplate.update("delete from person where id = ?", 5276L);
```

USING JdbcTemplate

eazy bytes

Other JdbcTemplate Operations

```
// You can use the execute(..) method to run any arbitrary SQL. Consequently, the method is often
// used for DDL statements.
this.jdbcTemplate.execute("create table person (id integer, name varchar(100))");
```

```
// The following example invokes a stored procedure
this.jdbcTemplate.update("call SUPPORT.REFRESH_PERSON_SUMMARY(?)",5276L);
```

USING ROWMAPPER

eazy bytes

- *RowMapper interface allows to map a row of the relations with the instance of user-defined class. It iterates the ResultSet internally and adds it into the collection. So we don't need to write a lot of code to fetch the records as ResultSetExtractor.*
- *RowMapper saves a lot of code becuase it internally adds the data of ResultSet into the collection.*
- *It defines only one method mapRow that accepts ResultSet instance and int as parameters. Below is the sample usage of it,*

```
private final RowMapper<Person> personRowMapper = (resultSet, rowNum) -> {
    Person person = new Person();
    person.setFirstName(resultSet.getString("first_name"));
    person.setLastName(resultSet.getString("last_name"));
    return person;
};
```

```
public List<Person> findAllPersons() {
    return this.jdbcTemplate.query("select first_name, last_name from person",personRowMapper);
}
```

QUICK TIP

eazy bytes

Do you know, if the column names in a table and field names inside a POJO/Bean are matching, then we can use BeanPropertyRowMapper which is provided by Spring framework.

Spring BeanPropertyRowMapper, class saves you a lot of time since we don't have to define the mappings like we do inside a RowMapper implementation.

```
public List<Holiday> findAllHolidays() {
    String sql = "SELECT * FROM HOLIDAYS";
    var rowMapper = BeanPropertyRowMapper.newInstance(Holiday.class);
    return jdbcTemplate.query(sql, rowMapper);
}
```

USING NamedParameterJdbcTemplate

eazy bytes

- *The NamedParameterJdbcTemplate class adds support for programming JDBC statements by using named parameters, as opposed to programming JDBC statements using only classic placeholder ('?') arguments. The NamedParameterJdbcTemplate class wraps a JdbcTemplate and delegates to the wrapped JdbcTemplate to do much of its work.*

- *The following example shows how to use NamedParameterJdbcTemplate*

```
private NamedParameterJdbcTemplate namedParameterJdbcTemplate;

public void setDataSource(DataSource dataSource) {
    this.namedParameterJdbcTemplate = new NamedParameterJdbcTemplate(dataSource);
}

public int countOfPersonsByFirstName(String firstName) {
    String sql = "select count(*) from Person where first_name = :first_name";
    SqlParameterSource namedParameters = new MapSqlParameterSource("first_name", firstName);
    return this.namedParameterJdbcTemplate.queryForObject(sql, namedParameters, Integer.class);
}
```

QUICK TIP

eazy bytes

Do you know, with Spring Boot working with JdbcTemplate is very easy. Spring Boot auto configures DataSource, JdbcTemplate and NamedParameterJdbcTemplate classes based on the DB connection details mentioned in the property file and you can @Autowire them directly into your own repository classes, as shown in the following example.

You can customize some properties of the template by using the `spring.jdbc.template.*` properties, like mentioned below,

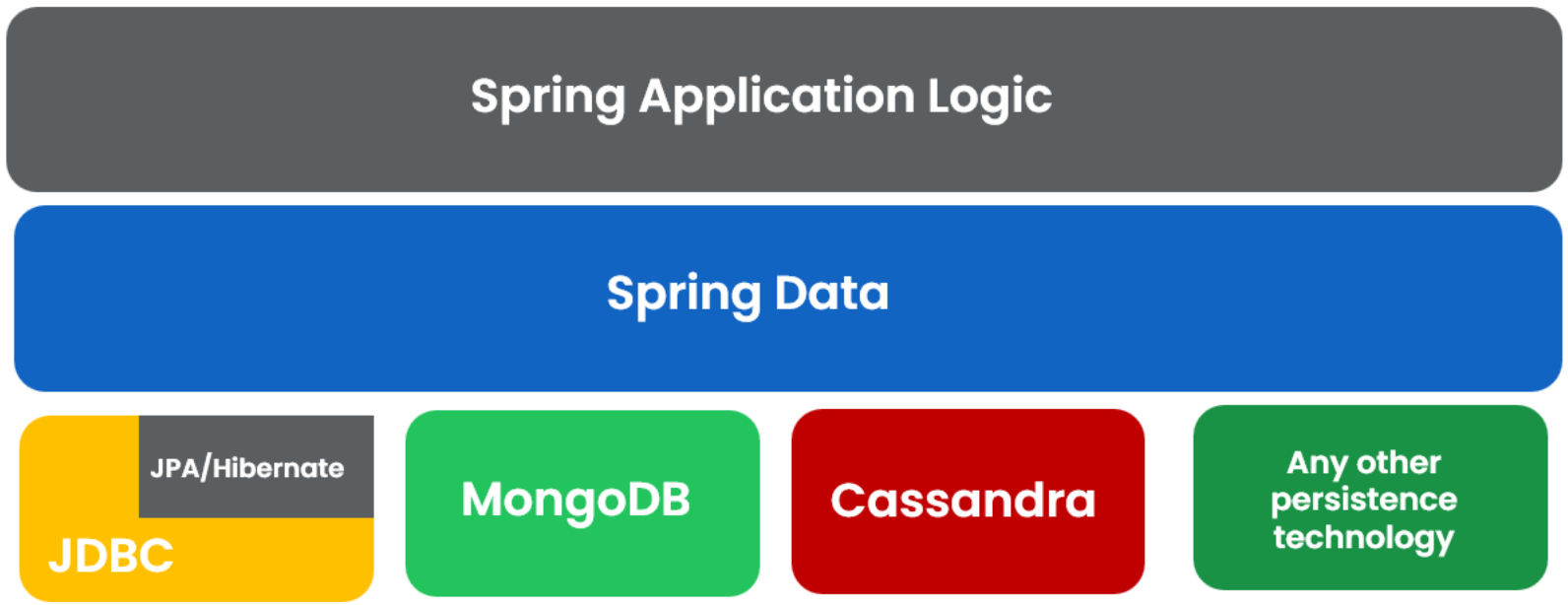
`spring.jdbc.template.max-rows=500`

```java
@Repository
public class HolidaysRepository {

    private final JdbcTemplate jdbcTemplate;

    @Autowired
    public HolidaysRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
```

INTRO TO SPRING DATA

eazy bytes

Spring Data is a Spring ecosystem project that simplifies the persistence layer's development by providing implementations according to the persistence technology we use. This way, we only need to write a few lines of code to define the repositories of our Spring app.



Spring Data is a high-level layer that simplifies the persistence implementation by unifying the various technologies under the same abstractions.

INTRO TO SPRING DATA

eazy bytes

- *Whichever persistence technology your app uses, Spring Data provides a common set of interfaces (contracts) you extend to define the app's persistence capabilities.*
- *The central interface in the Spring Data repository abstraction is Repository*

Repository is the most abstract contract. If you extend this contract, your app recognizes the interface you write as a particular Spring Data repository. Still, you won't inherit any predefined operations (such as adding a new record, retrieving all the records, or getting a record by its primary key). The Repository interface doesn't declare any method (it is a marker interface).

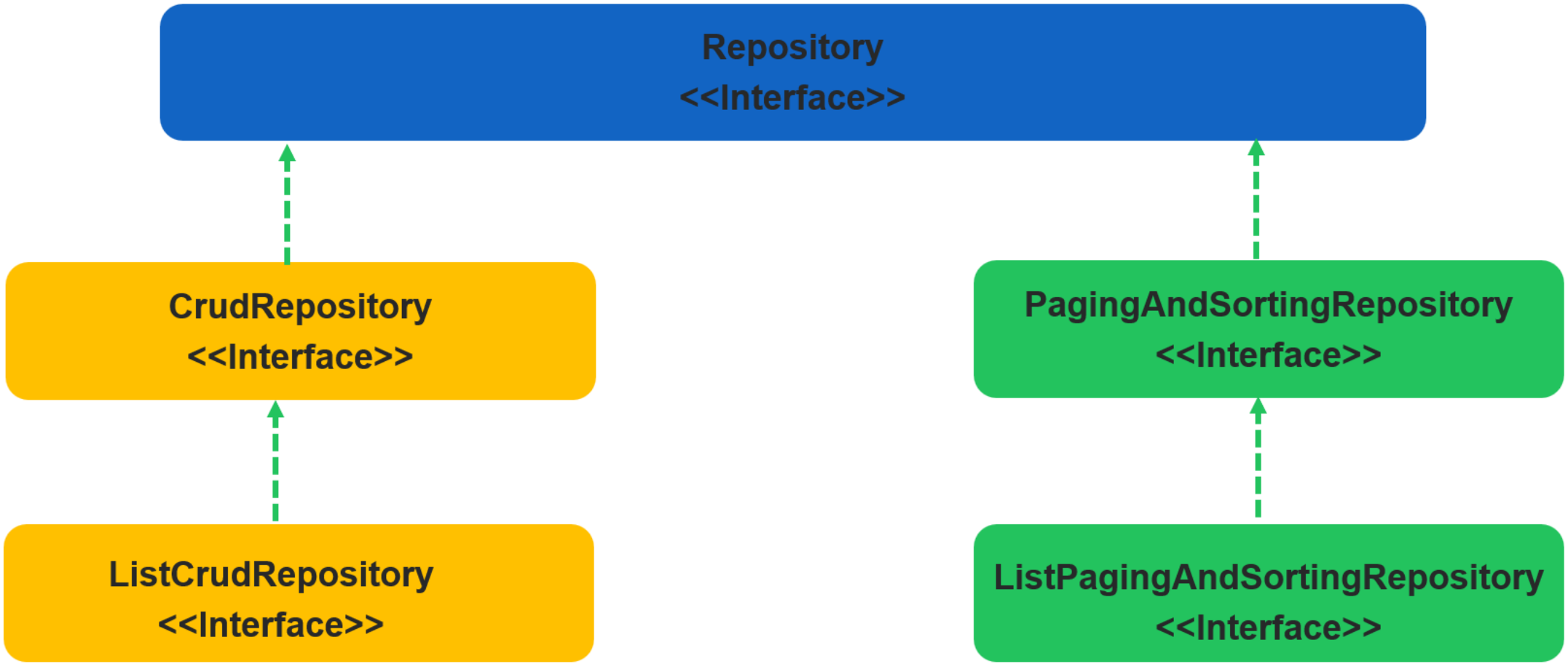
CrudRepository is the simplest Spring Data contract that also provides some persistence capabilities. If you extend this contract to define your app's persistence capabilities, you get the simplest operations for creating, retrieving, updating, and deleting records. **ListCrudRepository** is an extension to CrudRepository returning List instead of Iterable where ever applicable.

PagingAndSortingRepository provide methods to retrieve entities using the pagination and sorting abstraction. **ListPagingAndSortingRepository** an extension to PagingAndSortingRepository returning List instead of Iterable where ever applicable.

INTRO TO SPRING DATA

eazy bytes

To implement your app's repositories using Spring Data, you extend specific interfaces. The main interfaces that represent Spring Data contracts are Repository, CrudRepository, ListCrudRepository, PagingAndSortingRepository and ListPagingAndSortingRepository. You extend one of these contracts to implement your app's persistence capabilities.



QUICK TIP

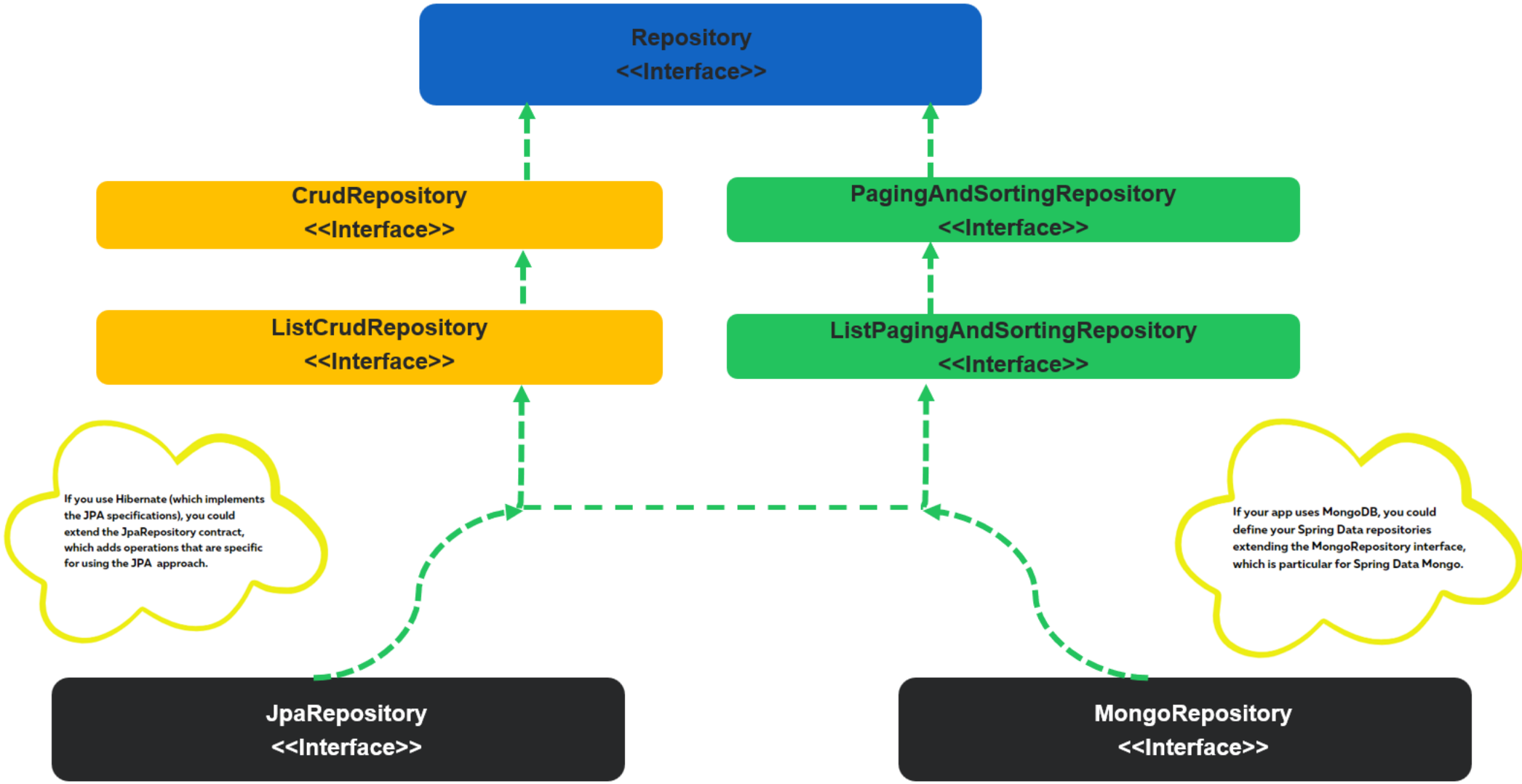
eazy bytes

Do you know,

- ✓ **We should not confuse between @Repository annotation and Spring Data Repository interface.**
- ✓ **Spring Data provides multiple interfaces that extend one another by following the principle called interface segregation. This helps apps to extend what they want instead of always following fat implementation.**
- ✓ **Some Spring Data modules might provide specific contracts to the technology they represent. For example, using Spring Data JPA, you also can extend the JpaRepository interface directly and similarly using Spring Data Mongo module to your app provides a particular contract named MongoRepository**

INTRO TO SPRING DATA

eazy bytes



INTRO TO SPRING DATA JPA

eazy bytes

- *Spring Data JPA is available to Spring Boot applications with the JPA starter. This starter dependency not only brings in Spring Data JPA, but also transitively includes Hibernate as the JPA implementation.*
- *Below is the maven dependency that we need to add to any SpringBoot projects in order to use Spring Data JPA,*

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

JPA is just a specification that defines an object-relational mapping (ORM) standard for storing, accessing, and managing Java objects in a relational database. Hibernate is the most popular and widely used implementation of JPA specifications. By default, Spring Data JPA uses Hibernate as a JPA provider.

- *We need to follow the below steps in order to query a DB using Spring Data JPA inside a Spring Boot applications,*

Step 1 : We need to indicate a java POJO class as an entity class by using annotations like @Entity, @Table, @Column

```
@Entity
@Table(name="contact_msg")
public class Contact extends BaseEntity{

    @Id
    @GeneratedValue(strategy= GenerationType.AUTO,generator="native")
    @GenericGenerator(name = "native",strategy = "native")
    @Column(name = "contact_id")
    private int contactId;
```

INTRO TO SPRING DATA JPA

eazy bytes

Step 2 : We need to create interfaces for a given table entity by extending framework provided Repository interfaces. This helps us to run the basic CRUD operations on the table w/o writing method implementations.

```
@Repository
public interface ContactRepository extends CrudRepository<Contact, Integer> {


}
```

Step 3 : Enable JPA functionality and scanning by using the annotations @EnableJpaRepositories and @EntityScan

```
@SpringBootApplication
@EnableJpaRepositories("com.eazybytes.eazyschool.repository")
@EntityScan("com.eazybytes.eazyschool.model")
public class EazyschoolApplication {
```

INTRO TO SPRING DATA JPA

eazy bytes

Step 4 : We can inject repository beans into any controller/service classes and execute the required DB operations.

```java
@Service
public class ContactService {

    @Autowired
    private ContactRepository contactRepository;

    public boolean saveMessageDetails(Contact contact){
        boolean isSaved = false;
        Contact savedContact = contactRepository.save(contact);
        if(null != savedContact && savedContact.getContactId()>0) {
            isSaved = true;
        }
        return isSaved;
    }
```

Derived Query Methods in Spring Data JPA

eazy bytes

- *With Spring Data JPA, we can use the method names to derive a query and fetch the results w/o writing code manually like with traditional JDBC*
- *As a developer we Just need to define the query methods in a repository interface that extends one of the Spring Data's repositories such as CrudRepository. Spring Data JPA will create queries and implementation at runtime automatically by parsing these method names.*
- *Below are few examples,*

```
// find persons by last name
List<Person> findByLastName(String lastName);
```

```
// find person by email
Person findByEmail(String email);
```

```
// find person by email and last name
Person findByEmailAndLastname(String email, String lastname);
```

QUICK TIP

eazy bytes

Do you know a derived query method name has two main components separated by the first By keyword.

1. The introducer clause like find, read, query, count, or get which tells Spring Data JPA what you want to do with the method. This clause can contain further expressions, such as Distinct to set a distinct flag on the query to be created.

2. The criteria clause that starts after the first By keyword. The first By acts as a delimiter to indicate the start of the actual query criteria. The criteria clause is where you define conditions on entity properties and concatenate them with And and Or keywords.

Using readBy, getBy, and queryBy in place of findBy will behave the same. For example, readByEmail(String email) is same as findByEmail(String email).

Derived Query Methods in Spring Data JPA

eazy bytes

Supported keywords inside method names

Keyword	Sample method name	Sample Query that form by JPA
Distinct	findDistinctByLastnameAndFirstname	select distinct … where x.lastname = ?1 and x.firstname = ?2
And	findByLastnameAndFirstname	… where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	… where x.lastname = ?1 or x.firstname = ?2
Is, Equals	findByFirstname,findByFirstnameIs,findBy FirstnameEquals	… where x.firstname = ?1
Between	findByStartDateBetween	… where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	… where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	… where x.age <= ?1
GreaterThan	findByAgeGreaterThan	… where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	… where x.age >= ?1
After	findByStartDateAfter	… where x.startDate > ?1
Before	findByStartDateBefore	… where x.startDate < ?1

Derived Query Methods in Spring Data JPA

eazy bytes

Supported keywords inside method names

Keyword	Sample method name	Sample Query that form by JPA
IsNull, Null	findByAge(Is)Null	… where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	… where x.age not null
Like	findByFirstnameLike	… where x.firstname like ?1
NotLike	findByFirstnameNotLike	… where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	… where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	… where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	… where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	… where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	… where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	… where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	… where x.age not in ?1

Derived Query Methods in Spring Data JPA

eazy bytes

Supported keywords inside method names

Keyword	Sample method name	Sample Query that form by JPA
True	findByActiveTrue()	… where x.active = true
False	findByActiveFalse()	… where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	… where UPPER(x.firstname) = UPPER(?1)

Spring Data JPA is a powerful tool that provides an extra layer of abstraction on top of an existing JPA providers like Hibernate. The derived query feature is one of the most loved features of Spring Data JPA.

Auditing Support By Spring Data JPA

eazy bytes

- *Spring Data provides sophisticated support to transparently keep track of who created or changed an entity and when the change happened. To benefit from that functionality, you have to equip your entity classes with auditing metadata that can be defined either using annotations or by implementing an interface.*
- *Additionally, auditing has to be enabled either through Annotation configuration or XML configuration to register the required infrastructure components.*
- *Below are the steps that needs to be followed,*

Step 1 : We need to use the annotations to indicate the audit related columns inside DB tables. Spring Data JPA ships with an entity listener that can be used to trigger the capturing of auditing information. We must register the AuditingEntityListener to be used for all the required entities.

```java
@Data
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public class BaseEntity {

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;
    @CreatedBy
    @Column(updatable = false)
    private String createdBy;
    @LastModifiedDate
    @Column(insertable = false)
    private LocalDateTime updatedAt;
    @LastModifiedBy
    @Column(insertable = false)
    private String updatedBy;
}
```

@CreatedDate, @CreatedBy, @LastModifiedDate, @LastModifiedBy are the key annotations that support JPA auditing

Auditing Support By Spring Data JPA

eazy bytes

Step 2 : Date related info will be fetched from the server by JPA but for CreatedBy & UpdatedBy we need to let JPA know how to fetch that info by implementing AuditorAware interface like shown below,

```
@Component("auditAwareImpl")
public class AuditAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
            return Optional.ofNullable(SecurityContextHolder.getContext()
                    .getAuthentication().getName());
    }
}
```

Step 3 : Enable JPA auditing by annotating a configuration class with the @EnableJpaAuditing annotation.

```
@SpringBootApplication
@EnableJpaRepositories("com.eazybytes.eazyschool.repository")
@EntityScan("com.eazybytes.eazyschool.model")
@EnableJpaAuditing(auditorAwareRef = "auditAwareImpl")
public class EazyschoolApplication {
```

QUICK TIP

eazy bytes

Do you know we can print the queries that are being formed and executed by Spring Data JPA by enabling the below properties,

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

- ✓ **show-sql property will print the query on the console/logs where as format_sql property will print the queries in a readable friendly style.**
- ✓ **But please make sure to leverage them in non-prod environments only as they impact the performance of the web application.**

Spring MVC Custom Validations

eazy bytes

We have seen before using Bean validations like Max, Min, Size etc. we can do validations on the input received. Now let's try to define custom validations that are specific to our business requirements. For the same we need to follow the below steps,

Step 1 : Suppose if we have a requirement to not allow some weak passwords inside our user registration form, we first need to create a custom annotation like below. Here we need to provide the class name where the actual validation logic present.

```
@Documented
@Constraint(validatedBy = PasswordStrengthValidator.class)
@Target( { ElementType.METHOD, ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface PasswordValidator {
    String message() default "Please choose a strong password";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Spring MVC Custom Validations

eazy bytes

Step 2 : We need to create a class that implements ConstraintValidator interface and overriding the isValid() method like shown below,

```
public class PasswordStrengthValidator implements
        ConstraintValidator<PasswordValidator, String> {

    List<String> weakPasswords;

    @Override
    public void initialize(PasswordValidator passwordValidator) {
        weakPasswords = Arrays.asList("12345", "password", "qwerty");
    }

    @Override
    public boolean isValid(String passwordField,
                           ConstraintValidatorContext cxt) {
        return passwordField != null && (!weakPasswords.contains(passwordField));
    }
}
```

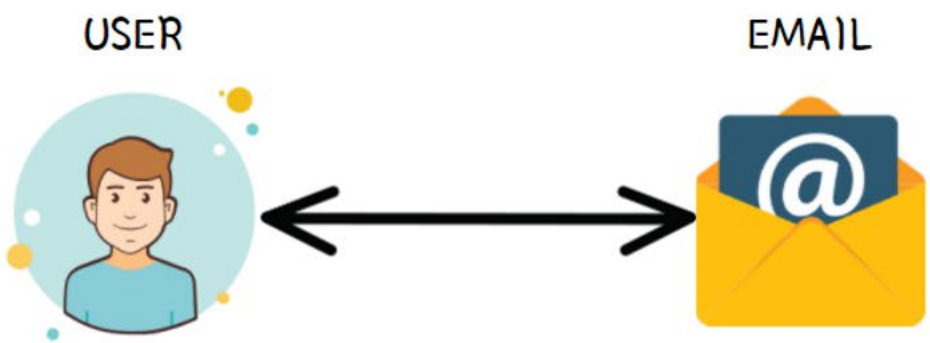
Step 3 : Finally we can mention the annotation that we created on top of the field inside a POJO class,

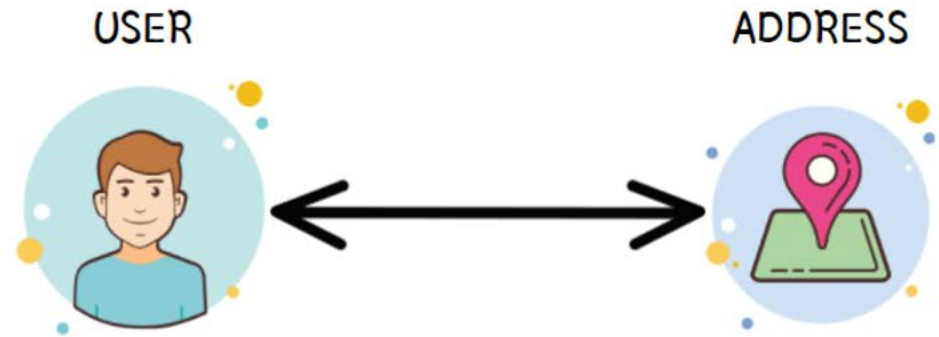
```
@NotBlank(message="Password must not be blank")
@Size(min=5, message="Password must be at least 5 characters long")
@PasswordValidator
private String pwd;
```

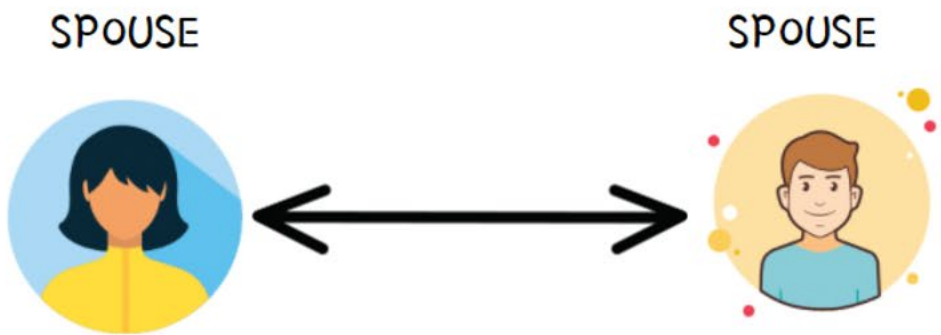
One to One Relationship inside JPA

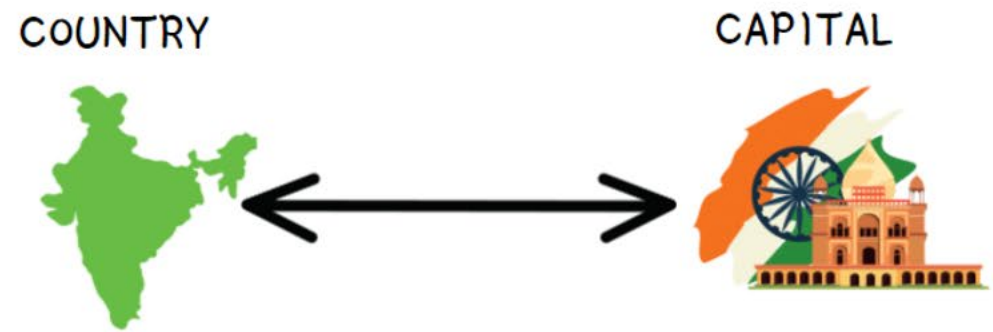
eazy bytes

- *First, what is a one-to-one relationship? It's a relationship where a record in one entity (table) is associated with exactly one record in another entity (table).*
- *Below are few real-life examples of one-to-one relationships,*









One to One Relationship inside JPA

eazy bytes

- *Spring Data JPA allow developers to build one-to-one relationship between the entities with simple configurations. For example if we want to build a one-to-one between Person and Address entities, then we can configure like mentioned below.*

```
@Data
@Entity
public class Person {

    @OneToOne(fetch = FetchType.EAGER,cascade = CascadeType.ALL, targetEntity = Address.class)
    @JoinColumn(name = "address_id", referencedColumnName = "addressId",nullable = true)
    private Address address;
}
```

- *In Spring Data JPA, a one-to-one relationship between two entities is declared by using the @OneToOne annotation. Using it we can configure FetchType, cascade effects, targetEntity.*
- *The @JoinColumn annotation is used to specify the foreign key column relationship details between the 2 entities. "name" defines the name of the foreign key column, referencedColumnName indicates the field name inside the target entity class, nullable defines whether foreign key column can be nullable or not.*

JPA FETCH

eazy bytes

- *Based on the fetch configurations that developer has done, JPA allows entities to load the objects with which they have a relationship.*

We can declare the fetch value in the @OneToOne, @OneToMany, @ManyToOne and @ManyToMany annotations. These annotations have an attribute called fetch that serves to indicate the type of fetch we want to perform. It have two valid values: FetchType.EAGER and FetchType.LAZY

With LAZY configuration, we are telling JPA that we want to lazily load the relation entities, so when retrieving an entity, its relations will not be loaded until unless we try refer the related entity using getter method. On the contrary, with EAGER it will load it's relation entities as well.

By default all ToMany relationships are LAZY, while ToOne relationships are EAGER.

eazy bytes

Key points of Cascade Types

Intro to Cascade

✓JPA allows us to propagate entity state changes from Parents to Child entities. This concept is calling Cascading in JPA.

✓The cascade configuration option accepts an array of CascadeTypes.

Cascade Types

The cascade types supported by JPA are as below:

1) CascadeType.PERSIST
2) CascadeType.MERGE
3) CascadeType.REFRESH
4) CascadeType.REMOVE
5) CascadeType.DETACH
6) CascadeType.ALL

Best Practices

✓ Cascading makes sense only for Parent – Child associations (where the Parent entity state transition being cascaded to its Child entities). Cascading from Child to Parent is not very useful and not recommended.

✓ There is no default cascade type in JPA. By default, no operation is cascaded.

CASCADE TYPES

eazy bytes

CascadeType.PERSIST : means that save() or persist() operations cascade to related entities.

CascadeType.MERGE : means that related entities are merged when the owning entity is merged.

CascadeType.REFRESH : means the child entity also gets reloaded from the database whenever the parent entity is refreshed.

CascadeType.REMOVE : means propagates the remove operation from parent to child entity.

CascadeType.DETACH : means detach all child entities if a “manual detach” occurs for parent.

CascadeType.ALL : is shorthand for all of the above cascade operations.

Spring Security AuthenticationProvider

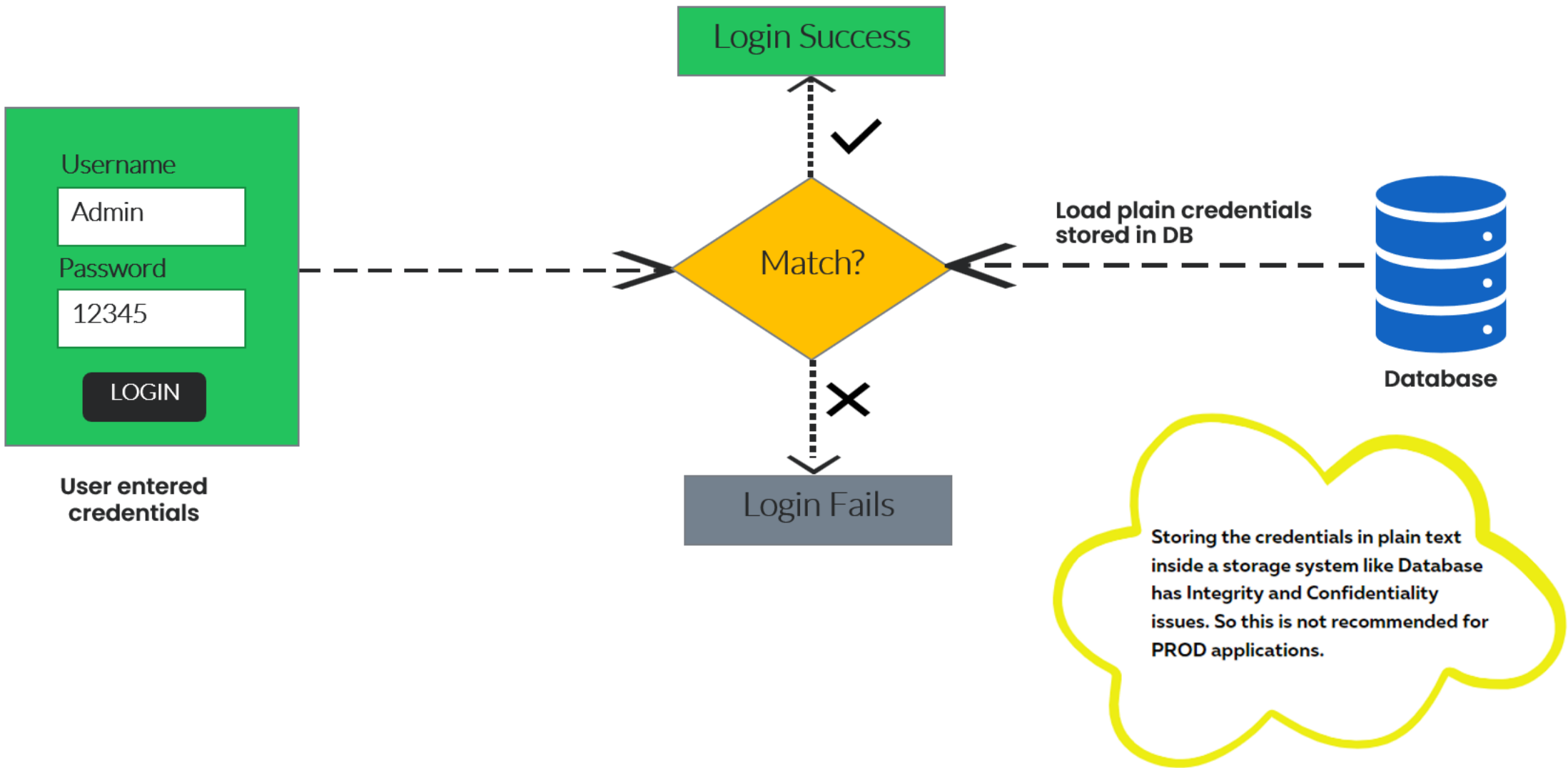
eazy bytes

- *As of now we are performing the login operation using the inMemoryAuthentication. But the idle way is to perform login check against a DB table or any other storage system which is more secure.*
- *For the same, Spring Security allow us to write our own custom logic to authenticate a user based on our requirements by implementing AuthenticationProvider interface. Below is the sample implementation of the same,*

```
@Component
public class EazySchoolUsernamePwdAuthenticationProvider implements AuthenticationProvider
{
    @Override
    public Authentication authenticate(Authentication authentication)
            throws AuthenticationException {
        String name = authentication.getName();
        String password = authentication.getCredentials().toString();
        if (authenticateUserBasedonYourRequirement()) {
            return new UsernamePasswordAuthenticationToken(
                    name, password, new ArrayList<>());
        } else {
            return null;
        }
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return authentication.equals(UsernamePasswordAuthenticationToken.class);
    }
}
```

How Passwords Validated W/O PasswordEncoder

eazy bytes



eazy bytes

Different ways of Pwd management

Encoding

- ✓ Encoding is defined as the process of converting data from one form to another and has nothing to do with cryptography.
- ✓ It involves no secret and completely reversible.
- ✓ Encoding can't be used for securing data. Below are the various publicly available algorithms used for encoding.

Ex: ASCII, BASE64, UNICODE

Encryption

- ✓ Encryption is defined as the process of transforming data in such a way that guarantees confidentiality.
- ✓ To achieve confidentiality,encryption requires the use of a secret which, in cryptographic terms, we call a "key".
- ✓ Encryption can be reversible by using decryption with the help of the "key". As long as the "key" is confidential, encryption can be considered as secured.

Hashing

- ✓ In hashing, data is converted to the hash value using some hashing function.
- ✓ Data once hashed is non-reversible. One cannot determine the original data from a hash value generated.
- ✓ Given some arbitrary data along with the output of a hashing algorithm, one can verify whether this data matches the original input data without needing to see the original data

QUICK TIP

eazy bytes

Do you know Spring Security provides various PasswordEncoders to help developers with hashing of the secured data like password.

Different Implementations of PasswordEncoders provided by Spring Security

- ✓ **NoOpPasswordEncoder (No hashing stores in plain text)**
- ✓ **StandardPasswordEncoder**
- ✓ **Pbkdf2PasswordEncoder**
- ✓ **BCryptPasswordEncoder (Most Commonly used)**
- ✓ **SCryptPasswordEncoder**

How Passwords Validated With Hashing &PasswordEncoder

eazy bytes